CodeAlpha Machine Learning Internship

TASK 1 — TECHNICAL REPORT

Credit Scoring Model

Decision Logic & Methodology

A complete walk-through of every technical decision taken in this project, with clear justification for each step.

Author Anas Mohamed

Dataset Kaggle — SyncoraAI Credit Scoring Dataset

Model Random Forest Classifier (tuned)

Date March 2026

Contents

1 Project Overview & Problem Definition **2**
1.1 Objective . . . 2
1.2 Task Type: Binary Classification . . . 2
1.3 Defining the Target Variable . . . 2
1.4 Columns Dropped Before Training . . . 2

2 Data Audit **3**
2.1 Why Audit First? . . . 3
2.2 Audit Results . . . 3
2.3 Class Imbalance Assessment . . . 3

3 Train / Validation / Test Split **3**
3.1 Why Split Before Anything Else? . . . 3
3.2 Split Strategy Chosen . . . 4

4 Exploratory Data Analysis **4**
4.1 Feature Structure . . . 4
4.2 Categorical Feature Inspection . . . 5

5 Preprocessing Pipeline **5**
5.1 Why a Unified Pipeline? . . . 5
5.2 Pipeline Architecture . . . 5

6 Model Selection **6**
6.1 Why Random Forest? . . . 6

7 Evaluation Strategy **7**
7.1 Why F1-Score, Not Accuracy? . . . 7
7.2 Confusion Matrix Results . . . 7

8 Model Interpretability **7**
8.1 Why Permutation Importance? . . . 7
8.2 Top 10 Features . . . 8

9 Hyperparameter Tuning **8**
9.1 Search Method: RandomizedSearchCV . . . 8
9.2 Search Space and Justification . . . 9

10 Data Leakage Compliance **9**

11 Conclusion **10**

1. Project Overview & Problem Definition

1.1. Objective

The goal of this project is to build a binary classification model that predicts whether a bank customer is likely to be a **bad credit risk**, using their financial history and spending behaviour as input features.

1.2. Task Type: Binary Classification

The output we want to predict is one of two values: good credit (0) or bad credit (1). This makes it a **binary classification** problem. This framing determines every downstream decision — which metrics are meaningful, how to handle class imbalance, and which evaluation strategy to use.

Because the classes are not perfectly balanced, **accuracy alone would be misleading**. A model that always predicts "good credit" would achieve high accuracy while completely failing its purpose. The primary metric is therefore **F1-Score**, which balances the cost of missing a bad credit customer against the cost of wrongly flagging a good one.

1.3. Defining the Target Variable

The raw dataset contains a continuous `CREDIT_SCORE` column, not a binary label. A threshold was needed to convert it into a classification target.

Decision: Threshold at 600 — Is_Bad_Credit

$$\texttt{Is_Bad_Credit} = \begin{cases} 1 & \text{if } \texttt{CREDIT_SCORE} < 600 \\ 0 & \text{otherwise} \end{cases}$$

A score below 600 is widely regarded in financial practice as "subprime" — indicating elevated lending risk. This threshold produces a meaningful, actionable label and aligns the model with real-world credit policy.

1.4. Columns Dropped Before Training

Three columns were identified as unsafe to include as features and were removed immediately:

Column	Reason Dropped	Category
`CUST_ID`	Unique row identifier, carries zero signal	Irrelevant
`CREDIT_SCORE`	Direct source of the target — including it would be cheating	Target Leakage
`DEFAULT`	Records whether a loan defaulted — information that only exists after the fact	Future Leakage

After removing these three columns and the derived label `Is_Bad_Credit`, **84 features** remain as model inputs.

2. Data Audit

2.1. Why Audit First?

Before any modelling decision is made, the data must be understood. The audit determines the appropriate model family, reveals whether imbalance handling is needed, and confirms there are no quality issues that would corrupt training.

2.2. Audit Results

Check	Finding	Action Taken
Dataset size	4,995 rows × 87 columns	Sufficient for standard ML
Feature types	51 float64, 34 int64, 2 object	Pipeline needed for mixed types
Duplicate rows	0	No action required
Missing values	0 across all 87 columns	No imputation strictly needed

2.3. Class Imbalance Assessment

The dataset has a moderate class imbalance — the majority of customers have good credit. When one class significantly outnumbers the other, a standard classifier will be biased toward the majority class, missing most bad credit cases.

Decision: class_weight = 'balanced'

Rather than generating synthetic minority samples (SMOTE), which adds pipeline complexity and must be carefully placed inside cross-validation to avoid leakage, `class_weight='balanced'` was chosen. It is built directly into `RandomForestClassifier`, re-weights the loss function automatically in proportion to class frequencies, and introduces no risk of information leakage.

3. Train / Validation / Test Split

3.1. Why Split Before Anything Else?

The split must happen before EDA statistics are computed, before scaling, and before any imputation. If preprocessing is fitted on the full dataset and then applied to splits, the model effectively "sees" validation and test data during training — producing optimistic, unreproducible scores.

Any statistic computed on the full dataset (e.g., mean for scaling, median for imputation) and then applied to train/test splits contaminates the model. The training set must be the only source of fitted statistics.

3.2. Split Strategy Chosen

Decision: Stratified Split — 70 / 15 / 15

Why stratified? A regular random split could concentrate most minority-class samples in one partition by chance. Stratification preserves the class ratio in every partition, ensuring each split is representative of the full dataset.
Why 70/15/15? With ≈ 5,000 rows, a 15% hold-out yields ≈ 750 validation samples — enough for stable metric estimates. Keeping 70% (≈ 3,500 rows) for training gives the Random Forest sufficient data to build reliable trees.
Test set discipline: The test set is untouched until after all hyperparameter decisions are finalised. Evaluating on test and then adjusting anything creates a model that is tuned to the test set — invalidating it as an unbiased estimator.

Split	Proportion	Rows	Purpose
Training	70%	3,497	Model fitting
Validation	15%	≈748	Metric evaluation and tuning
Test	15%	≈748	Final unbiased evaluation

The nested split was implemented with two successive stratified calls:

1. Extract the 15% test set from the full dataset.
2. Split the remaining 85% into 82.4% train / 17.6% val, producing the target 15% validation share of the original data.

4. Exploratory Data Analysis

4.1. Feature Structure

After leakage removal, 84 features remain:

- **82 numeric features** — raw financial values (income, debt, savings) and spending ratios across 12 categories (e.g. `R_DEBT_INCOME`, `T_UTILITIES_12`, `R_HEALTH_SAVINGS`).
- **2 categorical features** — `CAT_GAMBLING` and `CAT_DEBT`, encoded as object columns.
- **0 missing values** — confirmed on the training partition.

4.2. Categorical Feature Inspection

`CAT_GAMBLING` was inspected to determine the correct encoding strategy:

Category	Count (train)	Proportion
No	2,176	62.2%
High	884	25.3%
Low	437	12.5%

With only 3 distinct values, One-Hot Encoding is the appropriate choice. Label Encoding was rejected because it would impose a false numeric order (e.g. No=0, Low=1, High=2), implying that "High" is twice "Low" — which is not meaningful.

5. Preprocessing Pipeline

5.1. Why a Unified Pipeline?

All preprocessing steps and the model are encapsulated in a single `sklearn.pipeline.Pipeline` object. This guarantees that:

- Every transformer is fitted *only* on training data.
- Validation and test data are transformed using training statistics.
- The entire preprocessing logic travels with the model for deployment.
- Cross-validation automatically refits transformers per fold.

Without a pipeline, it is easy to accidentally fit a scaler on all data before splitting — one of the most common sources of inflated scores in ML projects.

5.2. Pipeline Architecture

A `ColumnTransformer` routes each feature type through its own branch:

Branch	Features	Steps Applied
Numeric	82 columns (int64 + float64)	`SimpleImputer(median)` → `StandardScaler`
Categorical	2 columns (object)	`SimpleImputer(most_frequent)` → `OneHotEncoder`

Decision: Why SimpleImputer (median) for numerics?

Although no missing values were found in this dataset, the imputer is included as a defensive production measure — unseen data may contain NaN values. The **median** strategy is preferred over the mean because many financial features (debt, income) are right-skewed; the median is more robust to extreme outliers and better represents the typical customer.

Decision: Why StandardScaler?

StandardScaler is included to make the pipeline forward-compatible. If a second model (e.g. Logistic Regression or SVM) is added for comparison later, no pipeline changes are needed — both require scaling. Random Forest itself does not need feature scaling, but including it has no negative effect on tree-based models.

Decision: Why OneHotEncoder for categorical columns?

The categorical features have low cardinality (3 values each). One-Hot Encoding creates one binary column per category with no implied ordering. `handle_unknown='ignore'` is set so that any unseen category in validation or test data is handled gracefully rather than raising an error.

6. Model Selection

6.1. Why Random Forest?

Several models were considered for this tabular binary classification task:

Consideration	Reasoning for Random Forest
Task specification	Random Forest is one of the algorithms explicitly listed in the CodeAlpha task requirements
Data size	4,995 rows is mid-range; RF is well-suited and does not require large data like deep learning
Mixed feature types	RF handles numeric and one-hot encoded features without additional tuning
Class imbalance	`class_weight='balanced'` is natively supported
Interpretability	Permutation importance is straightforward to extract from any RF pipeline
Robustness	RF is less sensitive to hyperparameter choices than gradient boosting, making it a reliable baseline

Decision: Baseline Configuration

- `n_estimators = 100` — 100 trees is the standard starting point. Enough trees for stable predictions without unnecessary computation.
- `class_weight = 'balanced'` — automatically corrects for class imbalance by upweighting the minority class during training.
- `random_state = 42` — ensures the experiment is fully reproducible.
- All other parameters left at sklearn defaults for a clean, unbiased baseline.

7. Evaluation Strategy

7.1. Why F1-Score, Not Accuracy?

In a class-imbalanced dataset, accuracy is deceptive. A trivial model that always predicts "good credit" would achieve high accuracy simply by exploiting the majority class, while being completely useless for the actual problem.

Decision: F1-Score as Primary Metric

F1-Score is the harmonic mean of Precision and Recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Both false positives (approving a risky borrower) and false negatives (rejecting a creditworthy customer) carry real costs. F1 penalises both equally and does not reward a model for exploiting class imbalance.
Precision and Recall are also reported separately, allowing a business stakeholder to shift the decision threshold if one type of error is more costly than the other.

7.2. Confusion Matrix Results

All evaluation was performed on the **validation set** (748 samples):

Model	TN	FP	FN	TP
Baseline (100 trees)	262	91	73	322
Tuned (best params)	264	89	72	323
Change	+2	**−2**	**−1**	+1

Derived performance metrics:

Metric	Baseline	Tuned
Accuracy	78.1%	78.5%
Precision (class 1)	78.0%	78.4%
Recall (class 1)	81.5%	81.8%
F1-Score (class 1)	79.7%	80.1%

8. Model Interpretability

8.1. Why Permutation Importance?

Two methods exist for extracting feature importance from a Random Forest:

Method	How It Works	Problem
MDI (Mean Decrease Impurity)	Built-in; measures impurity reduction at split nodes	Biased toward high-cardinality features
Permutation Importance	Shuffles each feature; measures score drop on validation	Unbiased; reflects true out-of-sample signal

Decision: Permutation Importance on the Validation Set

MDI is computed on training data, where the model has memorised patterns — features that were heavily used look artificially important. Permutation importance computed on the **validation set** reflects genuine out-of-sample predictive power. Shuffling a feature that truly matters causes a measurable drop in F1; shuffling an irrelevant feature does not.

8.2. Top 10 Features

Rank	Feature	Score	Interpretation
1	`R_DEBT_INCOME`	0.0821	Debt-to-income ratio
2	`R_UTILITIES_DEBT`	0.0111	Utilities spending vs. debt
3	`DEBT`	0.0099	Raw total debt amount
4	`R_EXPENDITURE_DEBT`	0.0080	Total spending vs. debt
5	`R_EXPENDITURE`	0.0044	Overall spending ratio
6	`R_TAX_DEBT`	0.0033	Tax payments vs. debt
7	`T_HEALTH_6`	0.0032	Health expenditure (6 months)
8	`R_HEALTH_SAVINGS`	0.0032	Health cost vs. savings
9	`T_HEALTH_12`	0.0025	Health expenditure (12 months)
10	`R_GROCERIES`	0.0024	Grocery spending ratio

Key insight: `R_DEBT_INCOME` is the dominant predictor with an importance score approximately 7× higher than the second feature. The top 10 are overwhelmingly *ratio* features — the relationship between debt and income or spending is the strongest signal for credit risk in this dataset.

9. Hyperparameter Tuning

9.1. Search Method: RandomizedSearchCV

Three search strategies were considered:

Method	Best For	Chosen?
Grid Search	Small grids with 1–2 parameters	No — 4 params, 96 combinations
Random Search	3–6 parameters with wide value ranges	**Yes**
Bayesian (Optuna)	Very expensive models with many parameters	Overkill for RF here

Decision: RandomizedSearchCV — 10 iterations, 3-fold CV, F1 scoring

Why Random over Grid? With 4 parameters and multiple values per parameter, an exhaustive grid produces 96 combinations. Random search samples 10 combinations and consistently finds near-optimal configurations at a fraction of the cost.
Why 3-fold CV? Cross-validation is run *inside* the search loop on the training set only. The validation set is reserved exclusively for final model evaluation and is never used to guide the hyperparameter search.
Why F1 scoring? The search optimises for the exact metric we care about in deployment, ensuring the best hyperparameters are selected for the right reason.

9.2. Search Space and Justification

Parameter	Values Searched	Why These Values
`n_estimators`	[100, 200, 300]	More trees reduce variance; 300 is a practical upper limit
`max_depth`	[None, 10, 20, 30]	None = fully grown; limiting depth regularises the model
`min_samples_leaf`	[1, 2, 4, 8]	The primary regulariser — larger values prevent memorising noise
`max_features`	['sqrt', 'log2']	Controls feature diversity per split; both are standard RF defaults

10. Data Leakage Compliance

Leakage is the single most dangerous and common mistake in ML projects. It produces models that appear to perform well but fail completely in production. Every decision in this project was made with leakage prevention as a primary constraint.

#	Check	Status
1	Data was split before any preprocessing or EDA statistics were computed	**PASS**
2	Imputers fitted on training set only — via Pipeline	**PASS**
3	Scalers fitted on training set only — via Pipeline	**PASS**
4	No target encoding was used — no leakage risk from encoding	**N/A**
5	Oversampling not used (class_weight used instead) — no fold leakage risk	**PASS**
6	No feature was derived directly from the target variable	**PASS**
7	`DEFAULT` column (post-loan outcome) dropped before training	**PASS**
8	`CREDIT_SCORE` (source of target) dropped before training	**PASS**
9	Permutation importance computed on validation set, not training set	**PASS**
10	Test set not evaluated until all hyperparameter decisions were final	**PASS**

11. Conclusion

This project built a complete, leak-proof credit scoring pipeline. Every technical decision — from the choice of threshold to define the target, to the evaluation metric, to the search strategy for tuning — was made with a clear rationale documented above.

- **F1-Score (Bad Credit):** 80.1%
- **Recall:** 81.8% — the model correctly identifies ≈4 in 5 bad credit customers
- **Precision:** 78.4%
- **Most predictive feature:** `R_DEBT_INCOME` (debt-to-income ratio)
- **Tuning gain:** 2 fewer false positives and 1 fewer false negative vs. baseline

This document was produced as part of the CodeAlpha ML Internship, Task 1.